

Contents

Contents	2
1 Modbus RTU communication.....	3
1.1 Serial port settings.....	3
1.2 Transmission mode	3
1.3 Data Types	3
1.4 Supported functions	4
1.5 Exceptions	4
1.6 Modbus/TCP	5
1.7 Network setting.....	5
1.8 NTP synchronization	5
2 Logical organization of the protection's data	7
2.1 Basic Data Type	7
2.1.1 BIT.....	7
2.1.2 BYTE.....	8
2.1.3 UBYTE	8
2.1.4 WORD.....	9
2.1.5 UWORD	9
2.1.6 LONG	10
2.1.7 ULONG	10
2.1.8 ARR.....	11
2.1.9 STR.....	12
2.1.10 CMD.....	12
3 How to read the Address Data Table.....	13
3.1 Address Data Table.....	13
3.1.1 VAR.....	13
3.1.2 REF	13
3.1.3 IDX	13
3.1.4 DIM.....	14
3.1.5 TYPE.....	14
3.1.6 UM.....	14
3.1.7 Kv14	
3.1.8 ENUM.....	15
3.2 Common properties.....	15
3.2.1 Input state	15
3.2.2 Output state.....	15
4 Examples	16
4.1 Example 1. How to read a variable	16
4.2 Example 2. How to read a BIT data type	17
4.3 Example 3. How to execute a command.....	17
5 Glossary	18
6 Appendix A. Address Data Table.....	19

Introduction

This Protocol Manual is for use with Thytronic Protective Relays, such as the DMC, NTG and PRO-N series that support Modbus RTU and Modbus TCP communication protocol.

1 Modbus RTU communication

The Thytronic Protective Relay (TPR) uses a communication protocol called Modbus. A company called Modicon, for use with their programmable controllers, developed the Modbus protocol. Since that time Modbus has evolved into common communication protocol in industry and it's now a "de-facto" standard.

The communication method involves using a master-slave technique, in which there is one master and several slaves. The TPR is a slave device. Only the master can initiate queries. These queries are directed to an individual slave device and the appropriate slave responds with the requested data.

There are two transmission modes. These modes are known as RTU (Remote Terminal Unit) and ASCII (American Standard Code for Information Interchange).

The TPR can be setup in a network of up to 247 slave devices. Each device must have a different address (1-247). The TPR can be set for RTU mode only.

→ The MODBUS protocol documentation can be found online at www.modicon.com or www.modbus.org (document "PDI-MBUS-300 REV J").

1.1 Serial port settings

To communicate with a TPR, the serial port of the Master has to be configured with the following settings:

Address	Baud Rate	Parity	Stop bits	Data bits
1	19200	No	1	8

Table 1 Serial port settings

Remark: Every TPR has a default MODBUS Slave Address set to 1.

1.2 Transmission mode

The transmission mode supported by TPR is RTU.

The ASCII transmission mode is not supported.

1.3 Data Types

A TPR has 4 data table, each of which corresponds to one of the 4 basic Modbus data type. Each table contains up to 1024 data that can be accessed with the proper function as described in the following table.

table/data type	reference	information	Access	Function Code for Reading	Function Code for Writing
Coils	0X	1 bit	Read/Write	0x01	0x05
Discrete Inputs	1X	1 bit	Read only	0x02	-
Input Registers	3X	16 bit Word	Read only	0x04	-
Holding Registers	4X	16 bit Word	Read/Write	0x03	0x06, 0x10

Table 2. Modbus Data Type

1.4 Supported functions

TPR supports the following Modbus functions:

Code	Name
0x01	Read Coil Status
0x02	Read Input Status
0x03	Read Holding Registers
0x04	Read Input Registers
0x05	Force Single Coil
0x06	Preset Single Register
0x0F	Force Multiple Coil
0x10	Preset Multiple Registers

Table 3 Supported Modbus functions

The smallest data that can be exchanged within a Modbus message is the Modbus Register. A Modbus Register is a 16 bit data in the Big Endian representation, called WORD (W)

$$\boxed{W} = \boxed{B_H} \boxed{B_L}$$

1.5 Exceptions

When a TPR receives a request that can not handle (e.g. the data address doesn't exist, the function is not supported, etc.), a special response called Exception is returned to the Master. This message contains the information needed to recognize the error occurred.

In an Exception message the most significant bit (msb) of the Function field is set to 1.

- **Function Code field:** in case of exception, the msb is set to 1.
- **Data field:** this field contains a value related to the kind of error generated by the request.

An example of request with exception response could be the following:

TX	04	07	A5	50	00	02	-	-
	S	F	ADR		DIM		CRC	
RX	04	87	01		-		-	
	S	F	DATA		CRC			

S = Slave Address
 F = Function Code
 ADR = Modbus Data Address
 DIM = Dimension (number of registers)
 DATA = data value
 CRC = Cyclical Redundancy Code

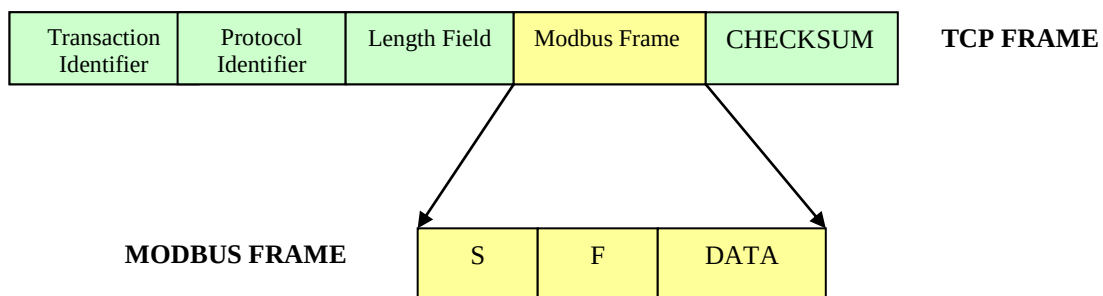
In this example the function code 07 is not supported by the TPR. The response is an exception message (the msb of function code F=0x87 is set to 1). The Data field value is 1 and according with the Modbus standard corresponds to the "Illegal function" exception.

The next table describes all the exceptions handled by the TPRs.

Code	Name	Description
01	ILLEGAL FUNCTION	The function code received in the query is not an allowable action for the slave.
02	ILLEGAL DATA ADDRESS	The data address received in the query is not an allowable address for the slave
03	ILLEGAL DATA VALUE	A value contained in the query data field is not an allowable value for the slave
04	SLAVE DEVICE FAILURE	An unrecoverable error occurred while the slave was attempting to perform the requested action.
05	ACKNOWLEDGE	The slave has accepted the request and is processing it, but a long duration of time will be required to do so. This response is returned to prevent a timeout error from occurring in the master.
06	SLAVE DEVICE BUSY	The slave is engaged in processing a long-duration program command. The master should retransmit the message later when the slave is free.

1.6 Modbus/TCP

Modbus/TCP, an extension of Modbus/RTU, defines how Modbus/RTU and Modbus/ASCII messages are encoded within and transported over TCP/IP-based networks. Modbus/TCP is just as simple to implement and flexible to apply as the original Modbus/RTU. The specification can be found online at www.modicon.com



1.7 Network setting

Every device connected to the TCP/IP network including the TPRs must have a unique IP address. Every TPR has the following default IP address:

Default IP Address = 200.1.1.135

1.8 NTP synchronization

To synchronize the Real Time Clock of more than one protection at the same time the NTP service is available. Network Time Protocol (NTP) is a protocol for synchronizing the clocks of computer systems over packet-switched, variable-latency data networks. NTP uses UDP port 123 as its transport layer. It is designed particularly to resist the effects of variable latency.

To synchronize all the TPR in a network is sufficient to send a broadcast NTP packet on the network.

→ The NTP service can be enabled/disabled changing the “NTP synchronization” parameter (Appendix A – Address Data Table).

More information about the NTP protocol and service can be found online at www.ntp.org

2 Logical organization of the protection's data

Each TPR has some data that can be read and/or written. These data are grouped in 4 data table.

Data with 1 bit of information (e.g. the state of an output), can be placed in the Discrete Inputs table or in the Coils table, while data with more than 1 bit of information (e.g. the value of a current), can be placed in the Register table, sometimes fitting more than one register as described in the following example.

1x, Discrete Input		3x, Input Registers	
n°	var	n°	var
6	I> Start	157	Power frequency
7	I> Trip	158	Phase current IL1 value – Word L
8	I> Block1	159	Phase current IL1 value – Word H
		160	Phase current IL2 value – Word L
		161	Phase current IL2 value – Word H

Table 4. Example of data structures

Example: let's read the value of the variable "I> Start". This variable is the first one in the Discrete Input data table. To read this kind of data the Modbus function 0x02 has to be used. As described in the standard the Modbus address to access the data is given by IDX-1 = 5 in this case.

To read the data from the Slave address 1 the next request has to be sent:

TX	01	02	00	05	00	01	-	-
	S	F	ADR		DIM		CRC	

2.1 Basic Data Type

In this chapter all the basic TPR data type and their relationship with the Modbus data type will be described.

2.1.1 BIT

Description:

Variable with 1 bit of information.

Relationship with Modbus:

The information corresponds to the least significant bit (lsb) of the first data byte addressed in the query.

2.1.2 BYTE

Description:

Signed variable with 1 byte (8 bit) of information. Range: -128 → +127

Relationship with Modbus:

A BYTE data type is always placed in the Least Significant Byte (LSB) of a Modbus Register (DIM=1).



Example:

relationship between the data BYTE whose value is 13 (0x0D) and its Modbus representation:



2.1.3 UBYTE

Description:

Unsigned variable with 1 byte (8 bit) of information. Range: 0 → +255

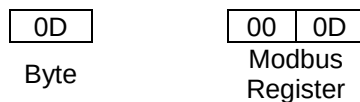
Relationship with Modbus:

A BYTE data type is always placed in the Least Significant Byte (LSB) of a Modbus Register (DIM=1).



Example:

relationship between the data BYTE whose value is 13 (0x0D) and its Modbus representation:



2.1.4 WORD

Description:

Signed variable with 2 byte (16 bit) of information. Range: -32.768 → +32.767

Relationship with Modbus:

A WORD data type is always placed in a Modbus Register (DIM=1) keeping the representation (Big Endian).



Example:

relationship between the data WORD whose value is 3073 (0x0C01) and its Modbus representation:



2.1.5 UWORD

Description:

Unsigned variable with 2 byte (16 bit) of information. Range: 0 → +65.535

Relationship with Modbus:

A UWORD data type is always placed in a Modbus Register (DIM=1) keeping the representation (Big Endian).



Example:

relationship between the data UWORD whose value is 3073 (0x0C01) and its Modbus representation:



2.1.6 LONG

Description:

Signed variable with 4 byte (32 bit) of information. Range: -2.147.483.648 → +2.147.483.647

Relationship with Modbus:

A LONG data type is placed in two Modbus Registers (DIM=2). The Least Significant Word of the LONG data type is placed in the first Modbus register while the Most Significant Word of the LONG data type is placed in the second Modbus register.



Example:

relationship between the data LONG 66536 (0x103E8) and its value in the Modbus protocol domain:



2.1.7 ULONG

Description:

Unsigned variable with 4 byte (32 bit) of information. Range: -0 → +4.294.967.295

Relationship with Modbus:

A ULONG data type is placed in two Modbus Registers (DIM=2). The Least Significant Word of the ULONG data type is placed in the first Modbus register while the Most Significant Word is placed in the second Modbus register.



Example:

relationship between the ULONG data whose value is 66536 (0x103E8) and its Modbus representation:



2.1.8 ARR

Description:

Array of BYTE.

Relationship with Modbus:

An ARR data type of length n is placed in N Modbus Registers ($N = \text{ceil}(n/2)$ [†]) keeping the byte ordering. If odd n , the last byte of ARR is placed in the MSB of the last Modbus register, while the LSB is set to 0

If even n :

B_1	B_2		B_n
1	2		n

Array[n]

B_1	B_2		B_{n-1}	B_n
1			N	

Modbus Registers

$$N = \text{ceil}(n/2)$$

If odd n :

B_1	B_2		B_n
1	2		n

Array[n]

B_1	B_2		B_n	0
1			N	

Modbus Registers

Example:

Let's consider an array with 3 bytes whose values are: (0x01, 0x3A, 0x1F). This array can be placed in $N = \text{ceil}(3/2) = \text{ceil}(1.5) = 2$ Modbus registers.

01	3A	1F
1	2	3

Array[3]

01	3A	1F	00
1		2	

Modbus Registers

[†] The *ceil* function returns the smallest integer value that is greater than or equal to a number. For example, $\text{ceil}(2.3) = 3$, $\text{ceil}(1.5) = 2$.

2.1.9 STR

Description:

String data type. It's represented by an array of BYTE. Every byte represents the hexadecimal ASCII code associated with the character of the string

Relationship with Modbus:

An STR data type of length n is placed in N Modbus Registers ($N = \text{ceil}(n/2)$ [†]) keeping the byte ordering. If odd n , the last byte is placed in the MSB of the last Modbus register. A zero padding operation (with char '\0') is necessary for every writing operation where the length of the string is lower than the number of bytes contained in the Modbus registers.

If even n :

C_1	C_2	...	C_n
1	2	...	n

String[n]

C_1	C_2	...	C_{n-1}	C_n
1			N	

Modbus Registers

$$N = \text{ceil}(n/2)$$

If odd n :

C_1	C_2	...	C_n
1	2	...	n

String[n]

C_1	C_2	...	C_n	'\0'
1			N	

Modbus Registers

Example:

Let's assume "Relay reference name" is a STRING variable whose length is 8 characters, its Modbus address is 0x0029 and its dimension is 4 Modbus registers. Let's set its value to "NAXx".

First of all the ASCII code in hexadecimal notation for each character of the string "NA10" is equal to 0x4E417878. An example of write request could be the following:

TX	01	10	00	29	00	05	10	4E	41	78	78	00	00	00	00	-	-
	S	F	ADR		DIM		B	DATA1		DATA2		DATA3		DATA4		CRC	
								'N'	'A'	'x'	'x'	'\0'	'\0'	'\0'	'\0'		

Remark. A zero padding operation is necessary because the length of the string "DMC901" is lower than the length of the RACK_TYPE variable.

2.1.10 CMD

Description:

This type of data represents a command.

Every Command corresponds to a Coil data type. To execute the command, the value of the associated Coil has to be set to 1 with the Modbus function 0x05.

[†] The *ceil* function returns the smallest integer value that is greater than or equal to a number. For example, $\text{ceil}(2.3)=3$, $\text{ceil}(1.5)=2$.

3 How to read the Address Data Table

All the information necessary to read/write data from/to a TPR is contained in the Address Data Table (see Appendix A).

3.1 Address Data Table

Now a description of all the Address Data Table's fields is given:

VAR	REF	IDX	DIM	TYPE	UM	KV	ENUM

Table 5. Address Data Table's field

3.1.1 VAR

The VAR field contains the description of the variable.

An example of VAR fields could be: "Relay phase nominal current - In ", "Relay reference name" or "I> Trip".

3.1.2 REF

The REF field tells which data table the variable is stored in.

The REF field can assume one of the following values:

- 0x: data stored in the COIL table
- 1x: data stored in the DISCRETE INPUT table
- 3x: data stored in the INPUT REGISTER table
- 4x: data stored in the HOLDING REGISTER table

3.1.3 IDX

The IDX field represents the position of the table (specified by REF) in which the variable VAR is stored.

The IDX value is related to the Modbus address of the variable. The relationship between IDX and Modbus address is the following:

$$\text{MB address} = \text{IDX} - 1$$

Example: let's consider the following Address Data Table

VAR	REF	IDX	DIM
Relay phase nominal current - In	3x	50	1
Relay reference name	3x	42	8
I> Trip	1x	7	1

Let's see that the variable "Relay phase nominal current - In" is the 50th variable stored in the Input Registers table (reference 3x) and its dimension is 1 Modbus register (DIM=1).

3.1.4 DIM

The DIM field is the number of Modbus registers necessary to contain the VAR data.

Example: let's read the value of the variable "Relay phase nominal current - In".

From the previous table we see that the reference is 4x for which the function 0x03 has to be used for reading.

The Modbus address is $IDX-1=50-1=49$.

The request for reading is the following:

TX	01	03	00	31	00	01	-	-
	S	F	ADR		DIM		CRC	

3.1.5 TYPE

The TYPE field specifies the basic data type associated with VAR, as described in chapter 2.1.

3.1.6 UM

The UM field specifies the Unit of Measure of the variable.

If UM is empty, it means that the Unit is just a number.

3.1.7 Kv

The Kv field contains the scale factor that has to be applied to the variable to obtain the real value. If Kv is not equal to 1, the value of the variable must be divided by Kv.

Example: let's consider the following address data table

VAR	REF	IDX	DIM	TYPE	UM	KV	ENUM
Phase current IL1 - Value	3x	159	2	LONG	In	16000	

Let's read the "Phase current IL1 - Value" from the Slave address 1.

TX	01	04	00	9E	00	02	-	-
	S	F	ADR		DIM		CRC	

Let's assume the following response:

RX	01	04	04	A9	80	00	03	-	-
	S	F	B	DATA1		DATA2		CRC	
				LONG					
				VAL _L		VAL _H			

The TYPE field is necessary to decode the right value returned from the Modbus response, as described in chapter 2.1. The decoded value contained in the response is $0x0003A980 = 240000$. $Kv=16000$ means that the value of the variable has to be divided by 16000. UM specify the Unit. The "Phase current IL1 - Value" is equal to 15 In.

3.1.8 ENUM

The ENUM field, whenever is defined, contains the meaning of all possible values assumed by the variable.

3.2 *Common properties*

Some common properties are now described for a better understanding.

3.2.1 Input state

- **State**
It's the physical state of the Input: 0 or 1 (OFF or ON)

3.2.2 Output state

- **State**
It's the logical state of the output: 0 or 1 (OFF or ON); could not correspond with its physical state
- **Fail**
It's the Coil Failure Status: 0 or 1 (OFF="OK" o ON="Fault").

4 Examples

For all the examples, the following symbols are assumed:

S = Slave Modbus address (1-247)
 F = Modbus Function code (1,2,3,4,5,6,15,16)
 B = number of byte in the DATA field
 ADR = Modbus DATA address
 DIM = number of Modbus DATA address
 CRC = Cyclical Redundancy Check
 DATAx = register x of DATA field
 VALx = value x of the variable

4.1 Example 1. How to read a variable

Let's read from the Slave address 1 the value of the "Relay phase nominal current - In".

From the Address Data Table (Appendix A) we find out the following information;

VAR	REF	IDX	DIM	TYPE	UM	Kv	ENUM
Relay phase nominal current - In	3x	50	1	UWORD	A	1	

The REF field value specify an *Input Register (3x)*, so the function 0x04 is used for reading. From the IDX the Modbus address is derived: Modbus address = IDX - 1 = 50 - 1 = 49. DIM specify the number of register to be read.

TX	01	04	00	3E	00	02	-	-
	S	F	ADR		DIM		CRC	
RX	01	04	04	00	05	-	-	
	S	F	B	DATA		CRC		

The TYPE field value is equal to UWORD. Applying the rules described in the chapter 2.1, the following value is obtained

$$VAL = 0x0005 = 5.$$

If UM field specify the Unit of measure that is "Ampere", while Kv = 1 means that the value doesn't need to be scaled. The "Relay phase nominal current - In" is equal to **5A**.

4.2 Example 2. How to read a BIT data type

Let's read from the Slave address 1 the variable "I> Trip".

From the Address Data Table (Appendix A) we find out the following information;

VAR	REF	IDX	DIM	TYPE	UM	Kv	ENUM
I> Trip	1x	7	1	BIT			0=OFF, 1=ON

The REF field value specify a *Discrete Input (1x)*, so the function 0x02 is used for reading. From the IDX the Modbus address is derived: Modbus address = $IDX - 1 = 7 - 1 = 6$. DIM specify the number of register to be read.

TX	01	02	00	06	00	01	-	-
	S	F	ADR		DIM		CRC	

RX	01	02	01	00	01	-	-
	S	F	B	VAL		CRC	

The TYPE field value is equal to BIT. Applying the rules described in the chapter 2.1, the following value is obtained

$$VAL = 1.$$

The ENUM field specifies the meaning of the value. In this case, the value 1 corresponds to the "ON" state and it means that a Trip has occurred on the first threshold of the 51 function.

4.3 Example 3. How to execute a command

Let's execute the command "Reset events" on the Slave address 1.

From the Address Data Table (Appendix A) we find out the following information;

VAR	REF	IDX	DIM	TYPE	UM	Kv	ENUM
Reset LEDs	0x	2	1	CMD			0=OFF, 1=ON

The TYPE field value specifies a *Command (CMD)*. As described in chapter 2.1, to execute the command, it is necessary to set its value to 1.

The REF field value specify a *Coil (0x)*, so the function 0x05 is used for writing. From the IDX the Modbus address is derived: Modbus address = $IDX - 1 = 2 - 1 = 1$

The Response message is known as an "Acknowledge" because it's an echo of the request, and it means that the Coil has been set:

TX	01	05	00	01	FF	00	-	-
	S	F	ADR		DATA		CRC	

RX	01	05	00	01	FF	00	-	-
	S	F	ADR		DATA		CRC	

5 Glossary

B _H	Most significative Byte of a Word data (Higher byte)
B _L	Least significative Byte of a Word data (Lower byte)
LSB	Least Significative Byte
lsb	Least significative bit
MSB	Most Significative Byte
msb	Most significative bit
NTP	Network Time Protocol
RTU	Remote Terminal Unit
TPR	Thytronic Protective Relay
W _H	Most significative Word of a Long data (Higher word)
W _L	Least significative Word of a Long data (Lower word)

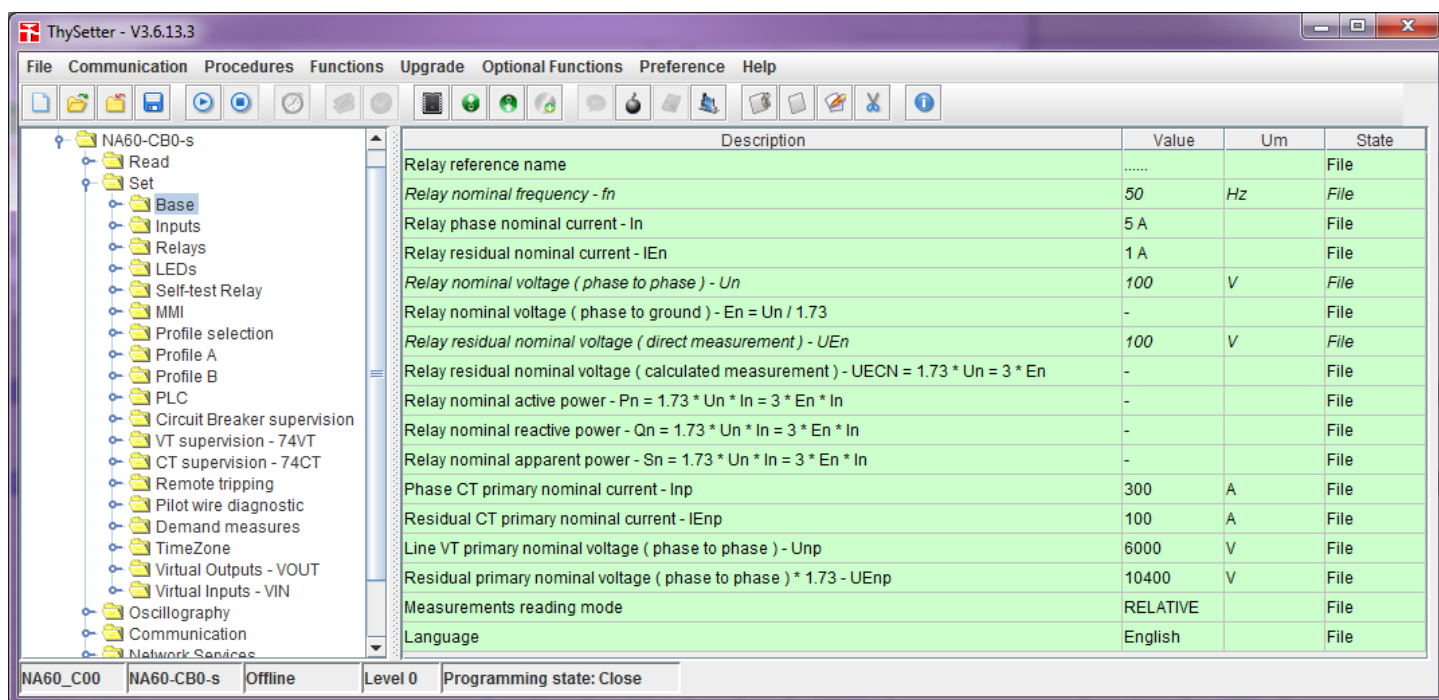
6 Appendix A. Practical Examples

In this appendix you will find some practical examples about how to convert the typical measure of a relay reading the relative registers.

Note that currents, voltages and powers are expressed in relative mode (In, Un, Pn) so if you need their absolute values in A, V, W you have to take in account the primary nominal value of the transformers.

All the values in the register has to be **divided** by the scale constant : "**Kv**"

So first of all take care of the datas in the : "Set/Base" menu of the relay. Ex:



Description	Value	Um	State
Relay reference name			File
Relay nominal frequency - fn	50	Hz	File
Relay phase nominal current - In	5 A		File
Relay residual nominal current - IEn	1 A		File
Relay nominal voltage (phase to phase) - Un	100	V	File
Relay nominal voltage (phase to ground) - En = Un / 1.73	-		File
Relay residual nominal voltage (direct measurement) - UEn	100	V	File
Relay residual nominal voltage (calculated measurement) - UECN = 1.73 * Un = 3 * En	-		File
Relay nominal active power - Pn = 1.73 * Un * In = 3 * En * In	-		File
Relay nominal reactive power - Qn = 1.73 * Un * In = 3 * En * In	-		File
Relay nominal apparent power - Sn = 1.73 * Un * In = 3 * En * In	-		File
Phase CT primary nominal current - Inp	300	A	File
Residual CT primary nominal current - IEnp	100	A	File
Line VT primary nominal voltage (phase to phase) - Unp	6000	V	File
Residual primary nominal voltage (phase to phase) * 1.73 - UEnp	10400	V	File
Measurements reading mode	RELATIVE		File
Language	English		File

TAB. 1

By the datas on tab. 1 you can extract:

fn= 50Hz
Inp= 300A
IEnp= 100A
Unp= 6000V (En=6000V/1.732=3464V)
UEnp= 10400V
PN=300A*6000V*1.732= 3.118MW

In the following table same measures as example.

VAR	REF	IDX	DIM	TYPE	Range	Um	Kv	ENUM
MEASURES								
Physical								
f - Value	3x	268	2	LONG		Hz	1000	
IL1 - Value	3x	270	2	LONG		In	16000	
IE - Value	3x	276	2	LONG		IEn	80000	
UL1 - Value	3x	278	2	LONG		En	112000	
UE - Value	3x	284	2	LONG		UEn	160000	
Calculated								
DTheta - Value	3x	286	2	LONG		DThetaB	1000	
U12 - Value	3x	288	2	LONG		Un	112000	
Displacement								
PhiL1 - Value	3x	310	2	LONG		deg	40	
Power								
P - Value	3x	334	2	LONG		Pn	172800	
Q - Value	3x	336	2	LONG		Qn	172800	
S - Value	3x	338	2	LONG		An	172800	
CosPhi - Value	3x	340	2	LONG			1000	
Energy measures								
EA+ - Value	3x	436	2	LONG		kWh	1	
EA- - Value	3x	438	2	LONG		kWh	1	
EA - Value	3x	440	2	LONG		kWh	1	
EQ+ - Value	3x	442	2	LONG		kvarh	1	
EQ- - Value	3x	444	2	LONG		kvarh	1	
EQ - Value	3x	446	2	LONG		kvarh	1	
PT100 Probes								
T1	3x	449	2	LONG		^C	10	

How to calculate the measure absolute values, starting from their modbus registers:

Frequency:

The frequency measure is a long so the value is composed by two (word) registers:

If we read on **f**:

Reg. 268= 0x0108 (low word)

Reg. 269= 0x0001 (high word)

Componing the two reg. we obtain : 0x00010108 (hexadecimal)= 65800 (dec)

Freq. measure= $65800/K_v = 65800/1000 = 65.8\text{Hz}$

Current:

The current measure is a long so the value is composed by two (word) registers:

If we read on **IL1**:

Reg. 270= 0x44C0 (low word)

Reg. 271= 0x0000 (high word)

Componing the two reg. we obtain : 0x000044C0 (hexadecimal)= 17600 (dec)

Current measure= $17600/K_v * I_{NP} = (17600/16000) * 300\text{A} = 330\text{A}$

Voltages:

The voltage measure is a long so the value is composed by two (word) registers:

If we read on **UL1**:

Reg. 278= 0xBE40 (low word)

Reg. 279= 0x0001 (high word)

Componing the two reg. we obtain : 0x0001BE40 (hexadecimal)= 114240 (dec)

Phase UL1 measure= $114240/K_v * E_{NP} = (114240/112000) * 3464\text{V} = 3533\text{V}$

The voltage measure is a long so the value is composed by two (word) registers:

If we read on **U12**:

Reg. 288= 0xBE40 (low word)

Reg. 289= 0x0001 (high word)

Componing the two reg. we obtain : 0x0001BE40 (hexadecimal)= 114240 (dec)

Phase U12 measure= $114240/K_v * U_{NP} = (114240/112000) * 6000\text{V} = 6120\text{V}$

Power:

The power measure is a long so the value is composed by two (word) registers:

If we read on **P**:

Reg. 334= 0x04EE (low word)

Reg. 335= 0x0003 (high word)

Componing the two reg. we obtain : 0x000304EE (hexadecimal)= 197870 (dec)

P measure= $197870/K_v * P_N = (197870/172800) * 3.118\text{MW} = 3.570\text{MW}$

Energy:

The power measure is a long so the value is composed by two (word) registers:

If we read on **EA**:

Reg. 440= 0x3DAD (low word)

Reg. 441= 0x0000 (high word)

Componing the two reg. we obtain : 0x00003DAD (hexadecimal)= 15789 (dec)

EA measure= $15789/K_v = (15789/1)=15789\text{kWh}$

Displacement:

The displacement measure is a long so the value is composed by two (word) registers:

If we read on **PhiL1**:

Reg. 310= 0x37F0 (low word)

Reg. 311= 0x0000 (high word)

Componing the two reg. we obtain : 0x000037F0 (hexadecimal)= 14320 (dec)

PhiL1 measure= $14320/K_v = 14320/40=358^\circ$

CosPhi:

The displacement measure is a long so the value is composed by two (word) registers:

If we read on **CosPhi**:

Reg. 340= 0x03D4 (low word)

Reg. 341= 0x0000 (high word)

Componing the two reg. we obtain : 0x000003D4 (hexadecimal)= 980 (dec)

CosPhi measure= $980/K_v = 980/1000=0.98$

Temperature:

The displacement measure is a long so the value is composed by two (word) registers:

If we read on **T1**:

Reg. 449= 0x0353 (low word)

Reg. 450= 0x0000 (high word)

Componing the two reg. we obtain : 0x00000353 (hexadecimal)= 851 (dec)

CosPhi measure= $851/K_v = 851/10=85.1^\circ\text{C}$